Cardflow Project

Members

1. Nontapat Auetrongjit 6833130321
2. Sira Tongsima 6833254521
3. Piyawat Jaikla 6833168221
4. Chadmongkol Tangwadthana 6833033121

2110215 Programming Methodology

Semester 2 Year 2025

Chulalongkorn University

Cardflow

Introduction

Cardflow is a puzzle/factory game inspired by Balatro, shapez.io and Mindustry. The objective of this game is to guide cards to different modifiers to eventually create a card that has the same value, suit and order as the requirement list in each level.

Gameplay Loop

The game operates in two total phases: the planning and simulation phase.

The Planning Phase

When a level begins, the game is paused in the Planning Phase. Here, the player can view the list of incoming and outgoing cards in the queue. They can also place and rotate *Movers* (filters and conveyors) onto the grid. The grid itself also comes with pre-placed *Modifiers* that can't be moved.

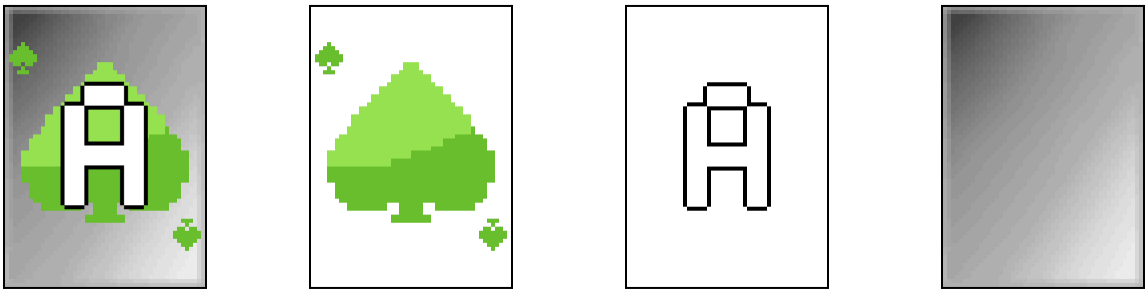
The Simulation Phase

Once the player decides that they are ready, they can begin the simulation phase. This is where the game runs and starts spawning cards into the grid. This will run until all the required cards reach the end or when the player stops the simulation manually.

Mechanics

1. Cards and Properties

Cards are the core objects of Cardflow. Every card possesses three properties in total: Suit, Value and Material

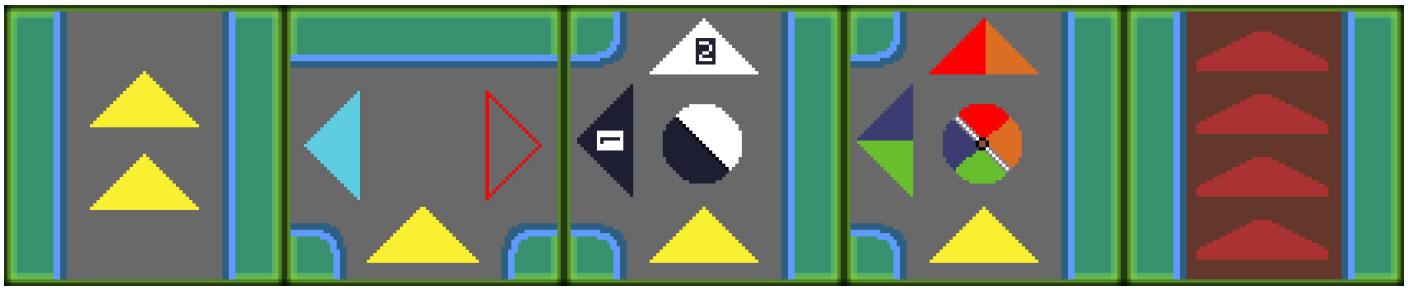


- **Suit:** The standard four suits (♠, ♥, ♦, ♣).
- **Value:** Values range from 1 to 13, where Face Cards are represented numerically (J = 11, Q = 12, K = 13). Arithmetic in Cardflow handles both overflow and underflow by wrapping around: subtracting 1 from an Ace (1) results in a King (13), and adding 1 to a King wraps back to an Ace.
- **Material:** The physical composition of the card determines how it reacts to Modifiers.
    - **Plastic:** The basic material with no special effects.
    - **Glass:** A fragile material that is destroyed after passing through 3 *modifiers*.
    - **Metal:** A sturdy material that is immune to *Arithmetic* (Add, Subtract, Multiply, Divide).
    - **Stone:** A dense material that is immune to *Combinators* (cannot be split, duplicated, merged, absorbed, or vaporized).

- **Rubber:** A bouncy material that causes all *changers* to apply their effect twice.
- **Corrupted:** A volatile material that permanently disables the very next *modifier* tile it passes through, after which it reverts to Plastic.

2. Movers

Movers are tools placed by the player during the Planning Phase. They will guide the cards along the grid. If a Mover attempts to push a card into an occupied tile, the movement is blocked until the space clears. If two movers attempt to push a card onto the same tile, it will prioritize movers coming from the top > left > right > bottom.



- **Conveyor:** Moves a card forward in its facing direction.
- **Flip-Flop:** Alternates its output direction (left - right) each time a card passes through it.
- **Parity Filter:** Analyzes the card's value. Even numbers pass straight forward while odd numbers are redirected to the left.
- **Red/Black Filter:** Analyzes the card's suit. Red suits (♥, ♦) pass straight forward while black suits (♠, ♣) are redirected to the left.
- **Delay:** It requires two movement ticks to release a card in its facing direction.

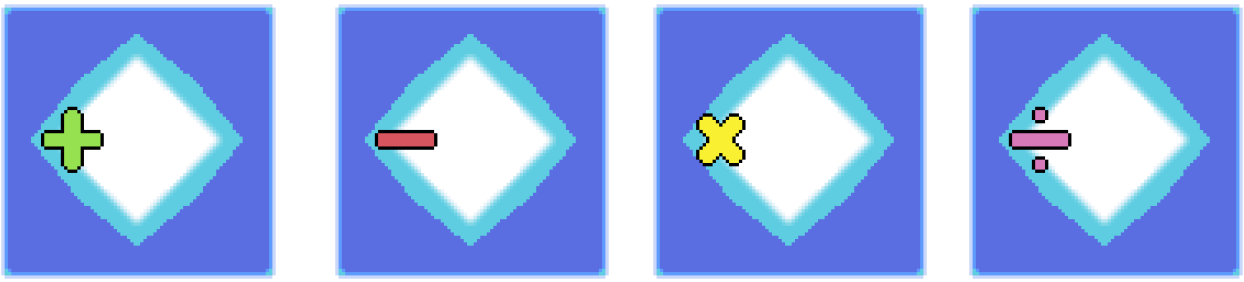
3. Modifiers

Modifiers are pre-placed tiles that alter a card's state when it rests on them. They are categorized into three distinct types: **Changers, Combinators** and **Pathways**.

3.1 Changers (Single-Card Modifiers)

Changers directly alter the properties of a single card in a single tick.

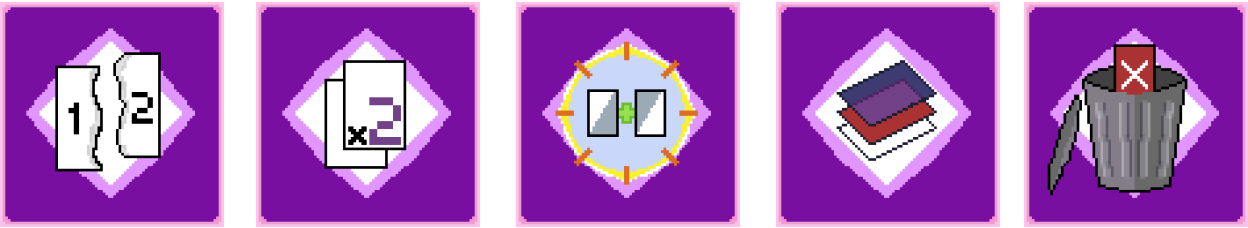
- **Arithmetic:** Perform basic math (Add, Subtract, Multiply, Divide) on the card's number.



- **Setters:** Overwrite a specific property, forcing the card to become a specific Number, Suit, or Material.

3.2 Combinators (Multi-Card / Multi-Tick Modifiers)

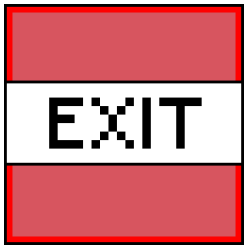
Combinators handle complex logic involving the destruction, creation, or merging of cards. Combinators can involve multiple cards and affect them on multiple ticks.



- **Splitter:** Divides a single card into two. The first card keeps half the numerical value (rounded down) and exits immediately. The second card (rounded up) is stored and released on the next tick.
- **Duplicator:** Functions on the same timing as the Splitter but produces an exact clone of the original card.
- **Merger:** Absorbs the first card that enters it, storing its value. When a second card enters, the stored value is added to the second card, and the new combined card exits.
- **Absorber:** Similar to the *Merger*, but instead of adding the values, the first card's number completely overwrites the second card's number.
- **Vaporizer:** Instantly and permanently destroys any card that enters it.

3.3 Pathways (Entrance, Exit)

Pathways handle the input and output of cards. These can't be disabled and they don't interact with a card's material. Each level will contain 1 Entrance and 1 Exit.



- **Entrance:** Spawns a card based on the level's input in the correct order.
- **Exit:** Removes a card and sends it for evaluation. It will only check for correctness of suit, value and order. The card's material does not matter.

4. The Tick System (Simulation Phase)

During the Simulation Phase, time moves in discrete "Ticks." The game will switch between the moving and modifying tick during the Simulation phase.

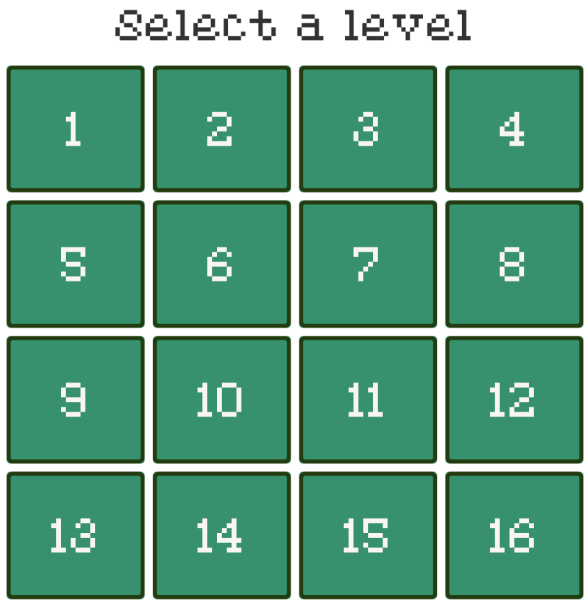
1. **Moving Tick:** All Movers on the board attempt to push their current cards to the next tile simultaneously.
2. **Modifying Tick:** All cards currently resting on an active Modifier tile apply their respective effects.

Gameplay

Main Menu

This is the game's start screen. It welcomes the player with the game title and lets them either start playing (Go to the level selector) or quit the app.

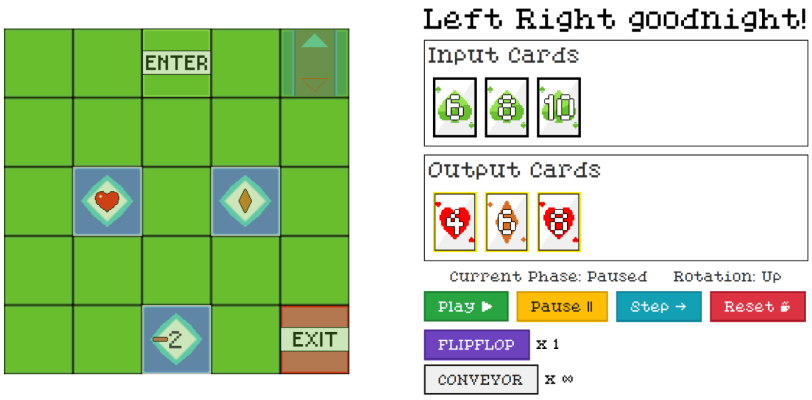
Level Select



This screen allows the player to choose which puzzle stage they want to play (1-16).

In Game

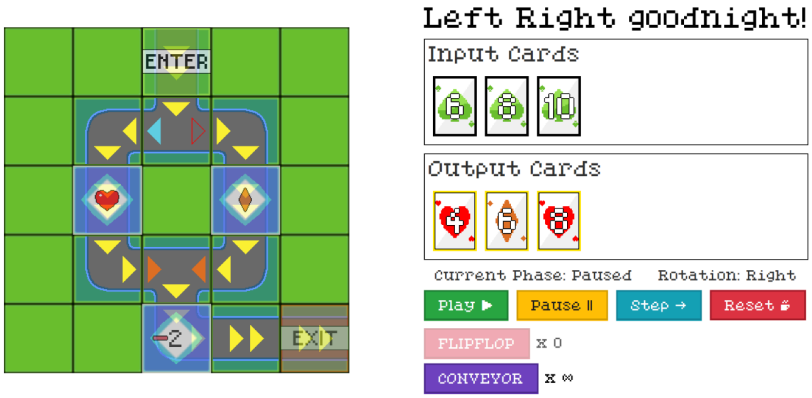
Starting a level



In the game screen there consists of two main components: the game grid and the level info.

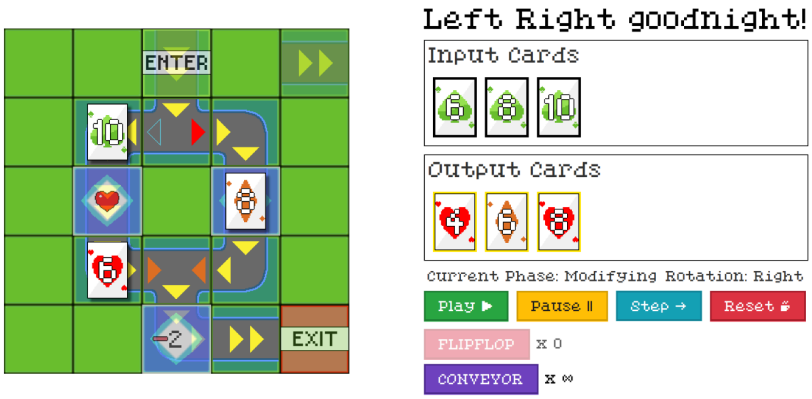
- **Game grid:** Contains tiles arranged in a grid. Each tile can contain a Modifier, a Card and a Mover. When hovering your mouse over a tile, your selected mover will show up as a shadow.
- **Level info:** Contains the level name, input card list, output card list, the current phase of the game (Paused, Modifying, Moving), the current rotation of your placement, tick control buttons (Play, Pause, Step and Reset) and the current player inventory.

Placement of movers



The current selected mover has a purple color. The player can place movers by clicking and dragging the movers in a line. The player can also rotate specific movers by right clicking the tile. The player can also rotate the placement rotation by right clicking anywhere on the screen as well. The player can change mover types by clicking on the button in the inventory. Once the player runs out of a specific type of mover, the inventory will show up as a muted red.

Simulating the cards



The player can use the tick control buttons to let the simulation play, pause the simulation, Step the simulation by tick or reset it and go back to the Planning phase.

Level win screen

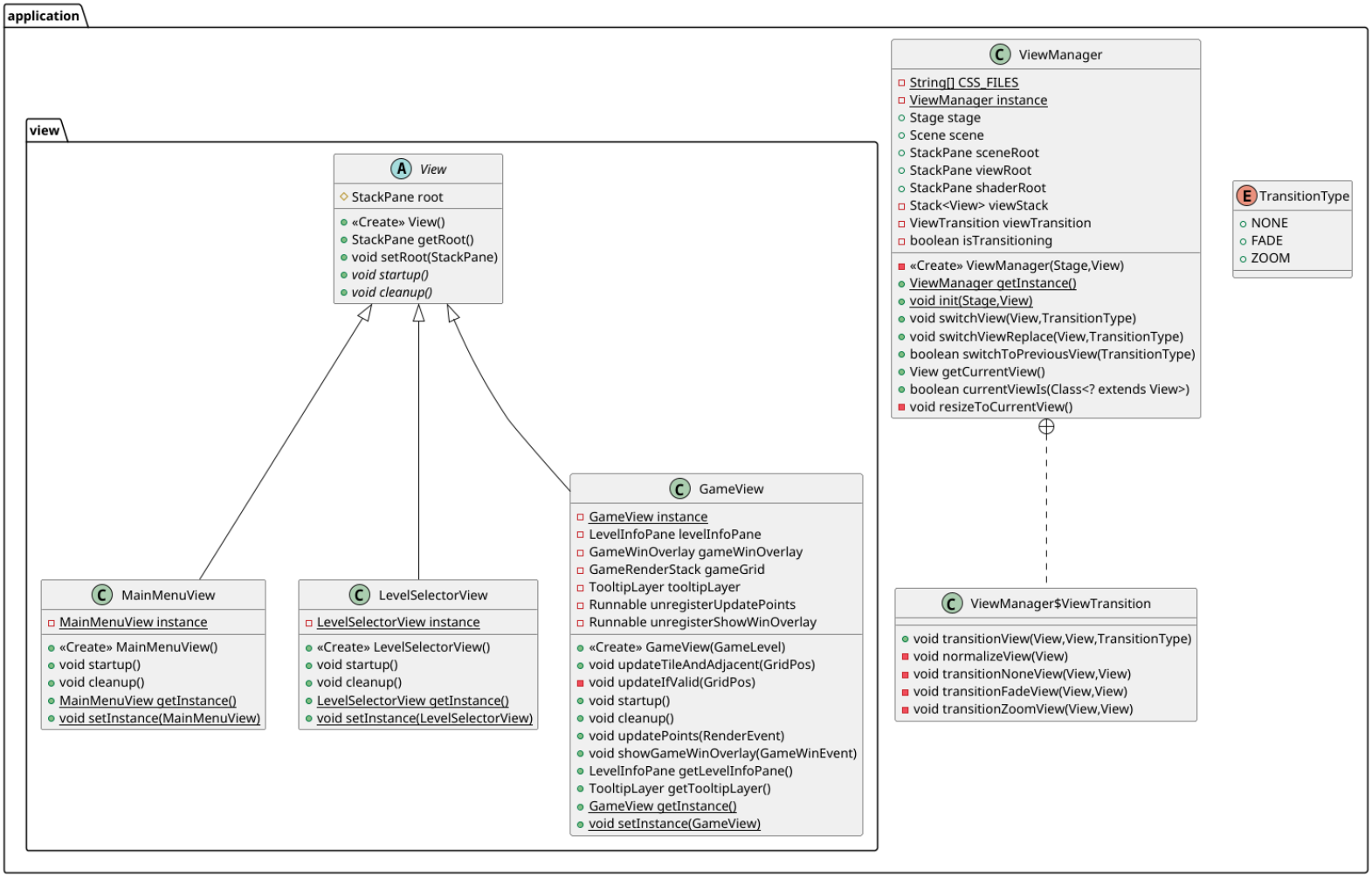
Once the player manages to bring cards to the exit in the correct suit value and order, the player will be greeted by an overlay allowing the player to go back to the level select menu or go to the next level.

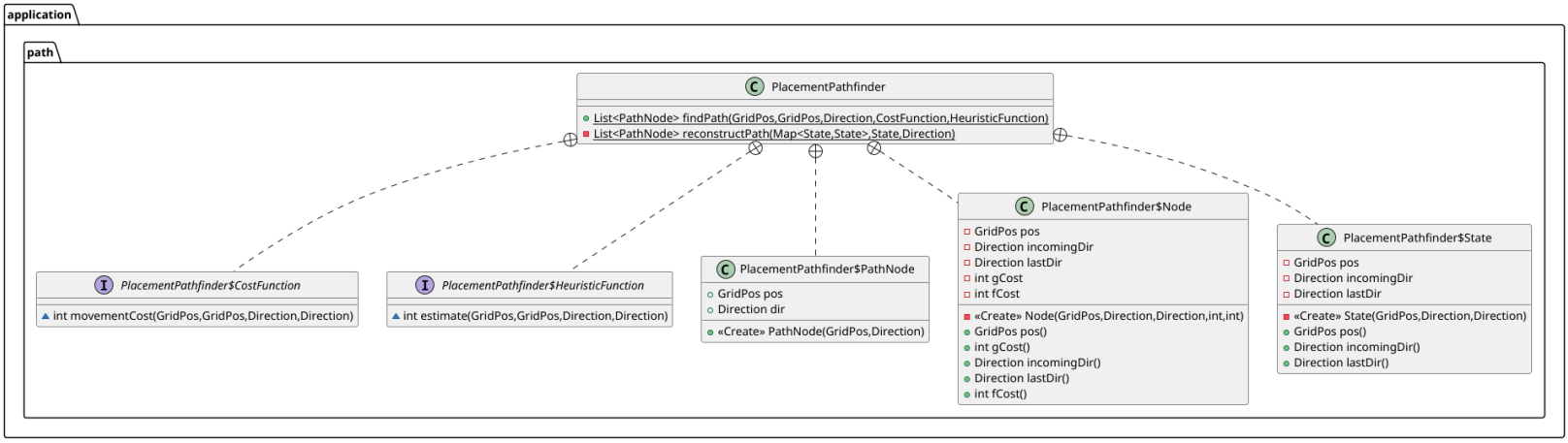
Javadoc

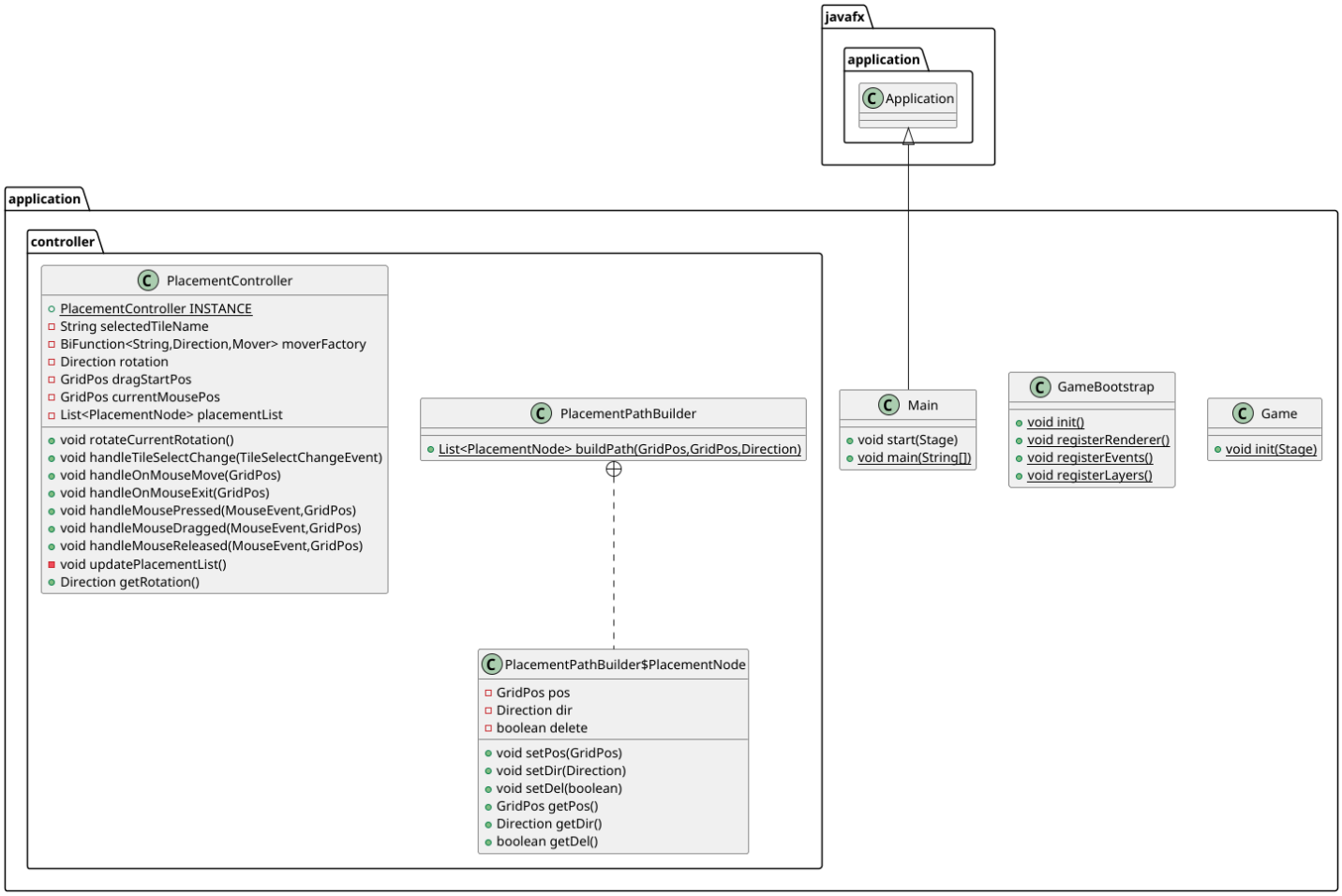
Inside sourcecode ./javadoc/index.html

UML Files

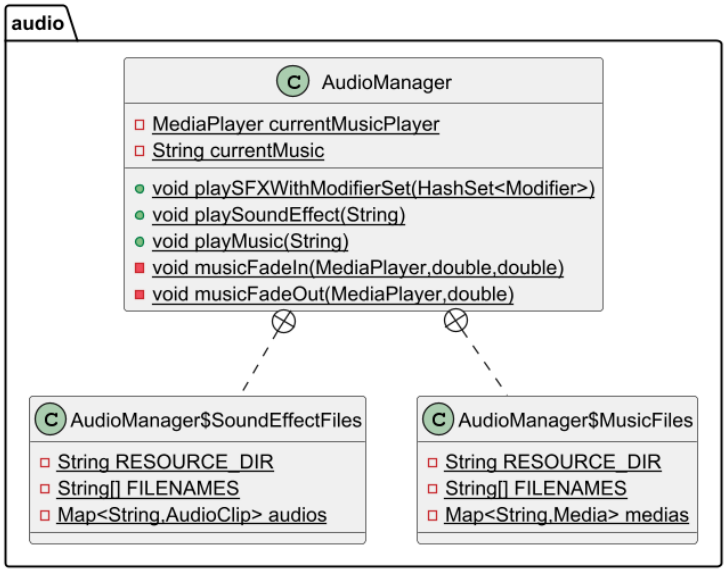
Application UML



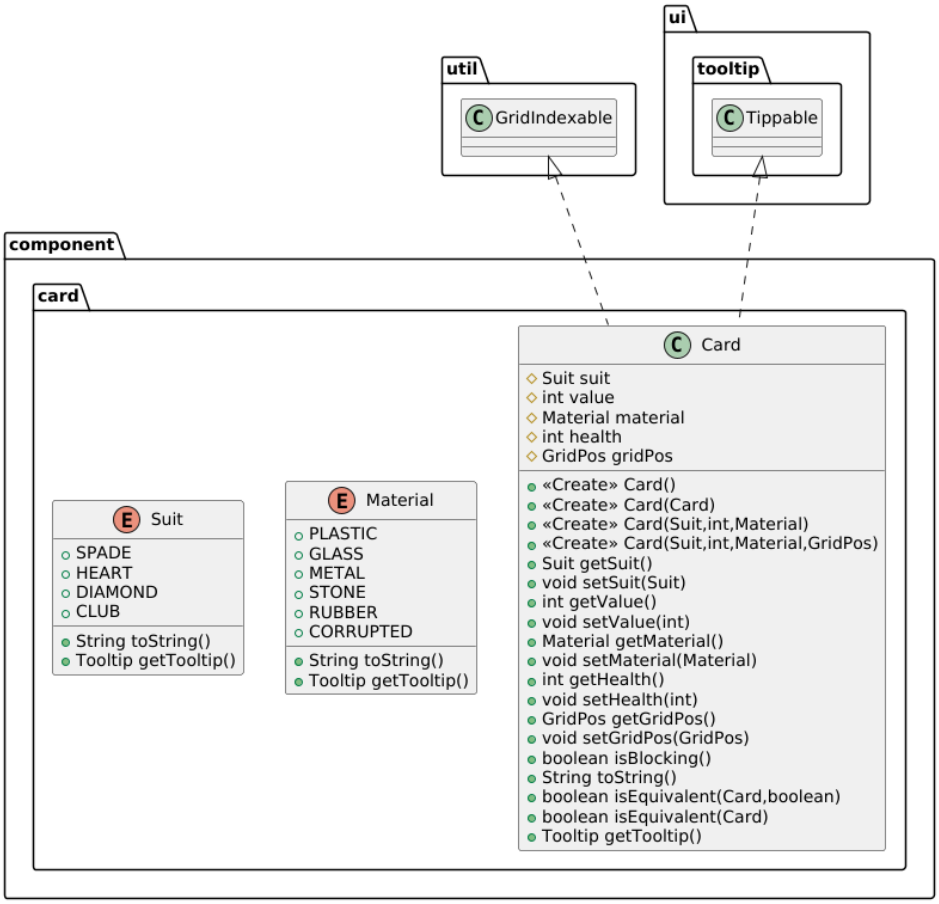



javafx
application
Application
application
controller
PlacementController
PlacementController INSTANCE
String selectedTileName
BiFunction<String,Direction,Mover> moverFactory
Direction rotation
GridPos dragStartPos
GridPos currentMousePos
List<PlacementNode> placementList
void rotateCurrentRotation()
void handleTileSelectChange(TileSelectChangeEvent)
void handleOnMouseMove(GridPos)
void handleOnMouseExit(GridPos)
void handleMousePressed(MouseEvent,GridPos)
void handleMouseDragged(MouseEvent,GridPos)
void handleMouseReleased(MouseEvent,GridPos)
void updatePlacementList()
Direction getRotation()
PlacementPathBuilder
List<PlacementNode> buildPath(GridPos,GridPos,Direction)
PlacementPathBuilder$PlacementNode
GridPos pos
Direction dir
boolean delete
void setPos(GridPos)
void setDir(Direction)
void setDel(boolean)
GridPos getPos()
Direction getDir()
boolean getDel()
Main
void start(Stage)
void main(String[])
GameBootstrap
void init()
void registerRenderer()
void registerEvents()
void registerLayers()
Game
void init(Stage)

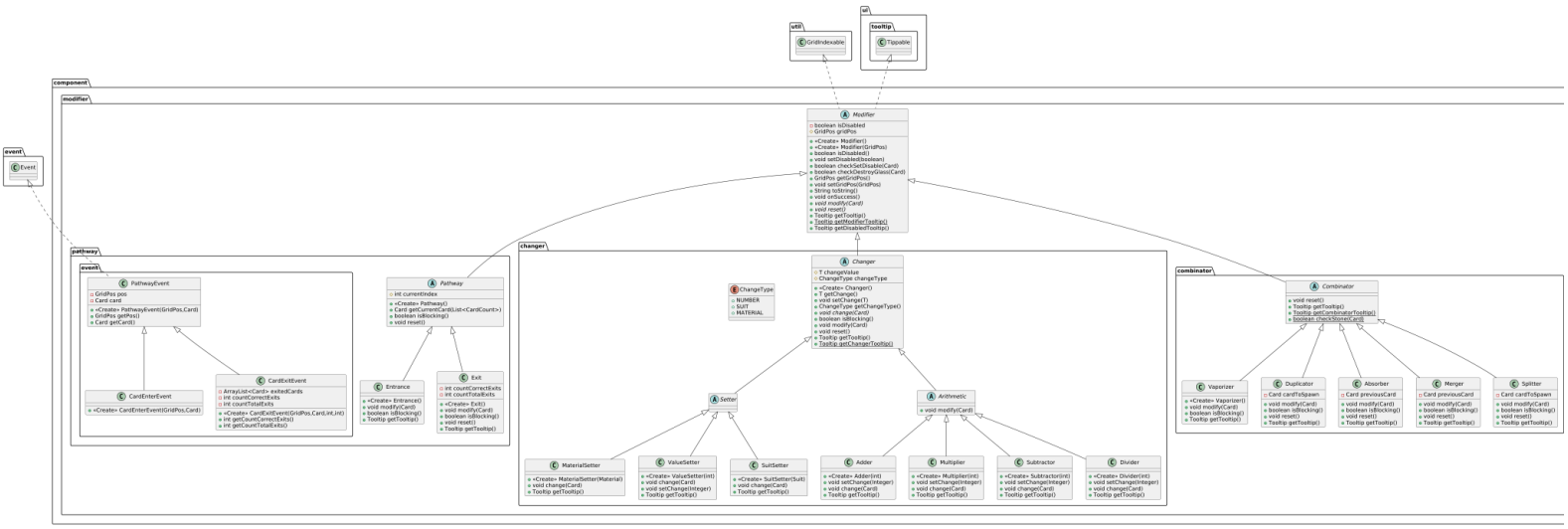
Audio UML

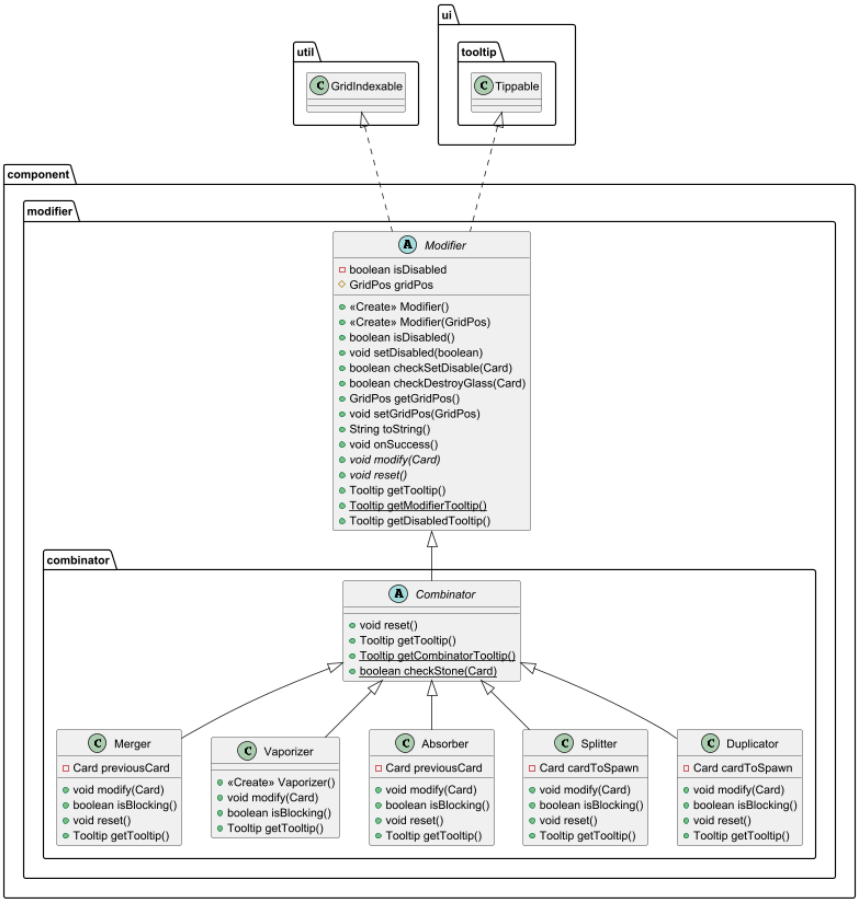

audio
AudioManager
MediaPlayer currentMusicPlayer
String currentMusic
void playSFXWithModifierSet(HashSet<Modifier>)
void playSoundEffect(String)
void playMusic(String)
void musicFadeIn(MediaPlayer,double,double)
void musicFadeOut(MediaPlayer,double)
AudioManager$SoundEffectFiles
String RESOURCE_DIR
String[] FILENAMES
Map<String,AudioClip> audios
AudioManager$MusicFiles
String RESOURCE_DIR
String[] FILENAMES
Map<String,Media> medias

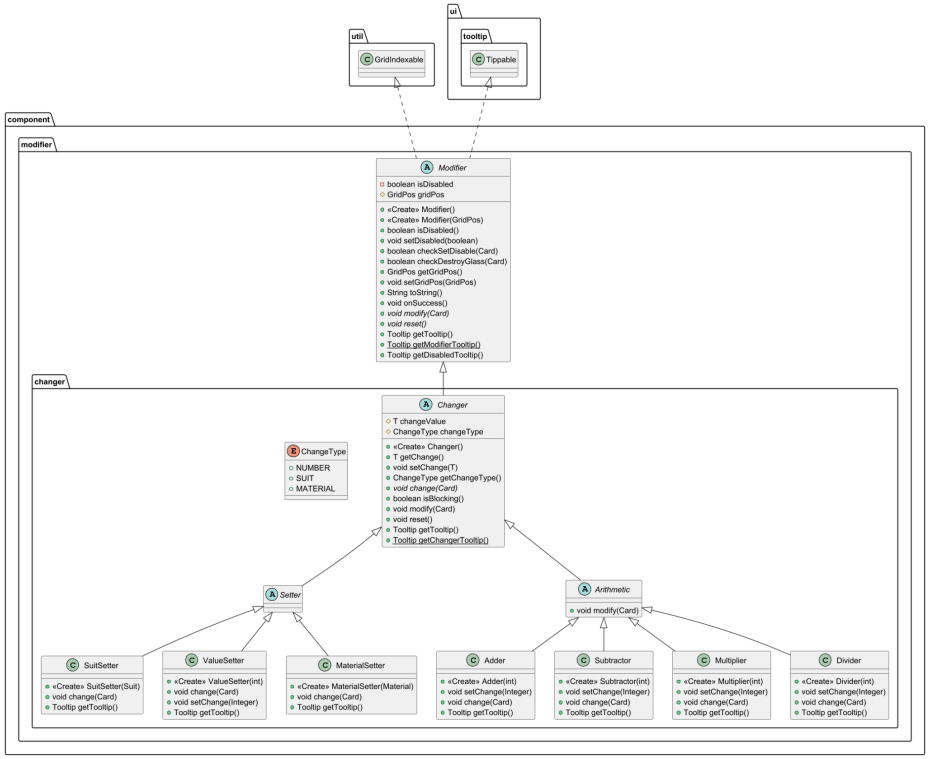
Component.Card UML

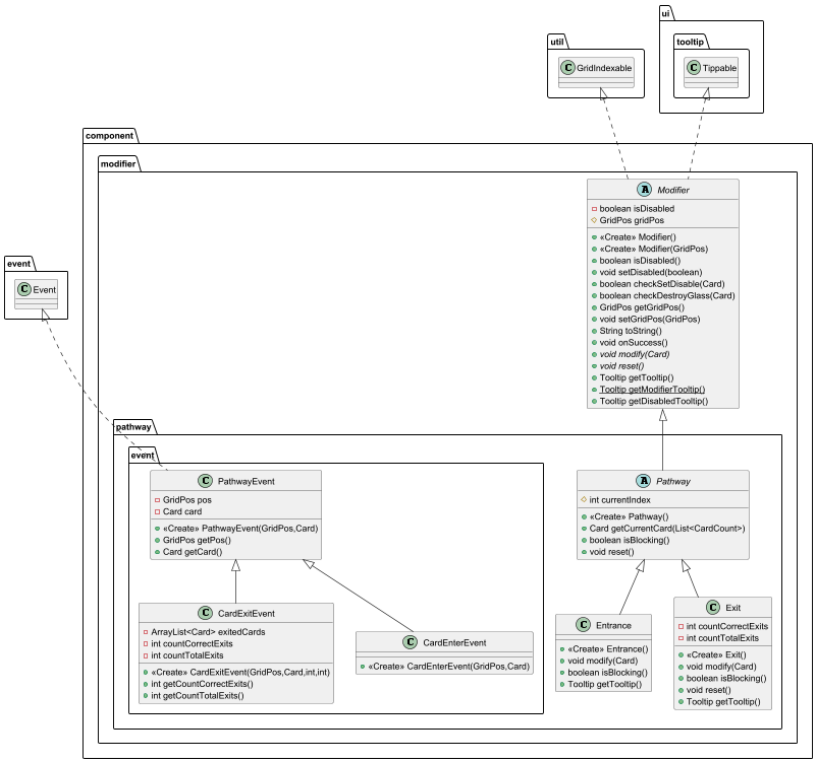


Component.Modifier

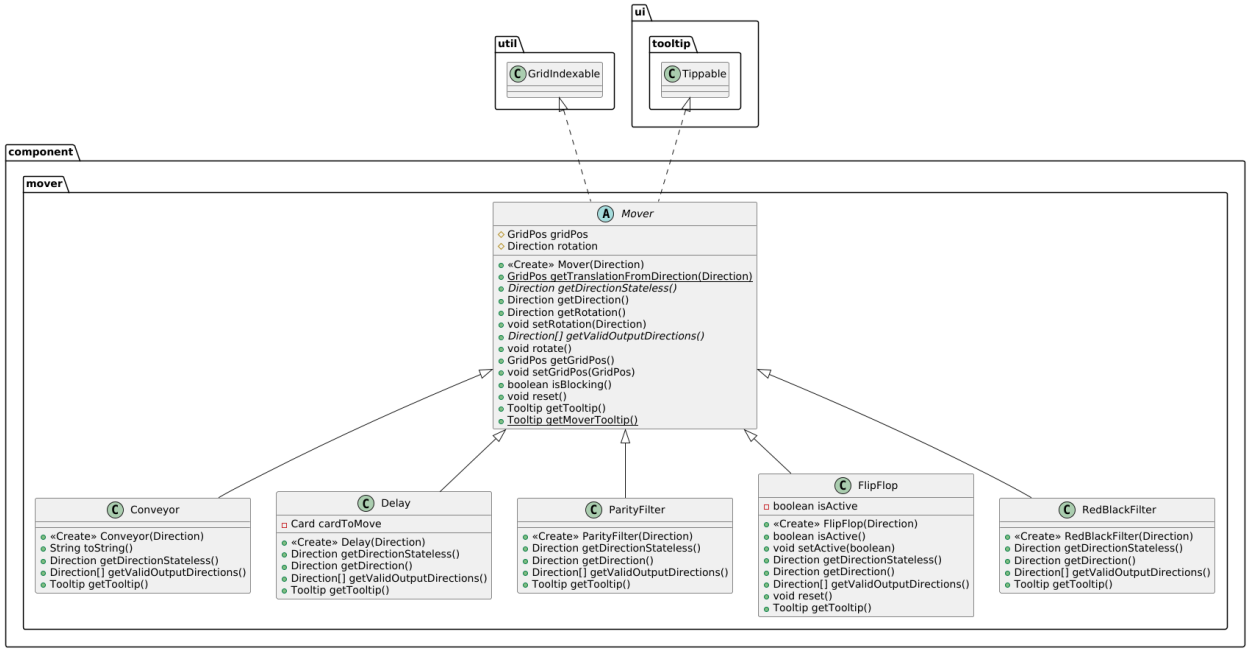



util
GridIndexable
ui
tooltip
Tippable
component
modifier
Modifier
boolean isDisabled
GridPos gridPos
«Create» Modifier()
«Create» Modifier(GridPos)
boolean isDisabled()
void setDisabled(boolean)
boolean checkSetDisable(Card)
boolean checkDestroyGlass(Card)
GridPos getGridPos()
void setGridPos(GridPos)
String toString()
void onSuccess()
void modify(Card)
void reset()
Tooltip getTooltip()
Tooltip getModifierTooltip()
Tooltip getDisabledTooltip()
combinator
Combinator
void reset()
Tooltip getTooltip()
Tooltip getCombinatorTooltip()
boolean checkStone(Card)
Merger
Card previousCard
void modify(Card)
boolean isBlocking()
void reset()
Tooltip getTooltip()
Vaporizer
«Create» Vaporizer()
void modify(Card)
boolean isBlocking()
Tooltip getTooltip()
Absorber
Card previousCard
void modify(Card)
boolean isBlocking()
void reset()
Tooltip getTooltip()
Splitter
Card cardToSpawn
void modify(Card)
boolean isBlocking()
void reset()
Tooltip getTooltip()
Duplicator
Card cardToSpawn
void modify(Card)
boolean isBlocking()
void reset()
Tooltip getTooltip()

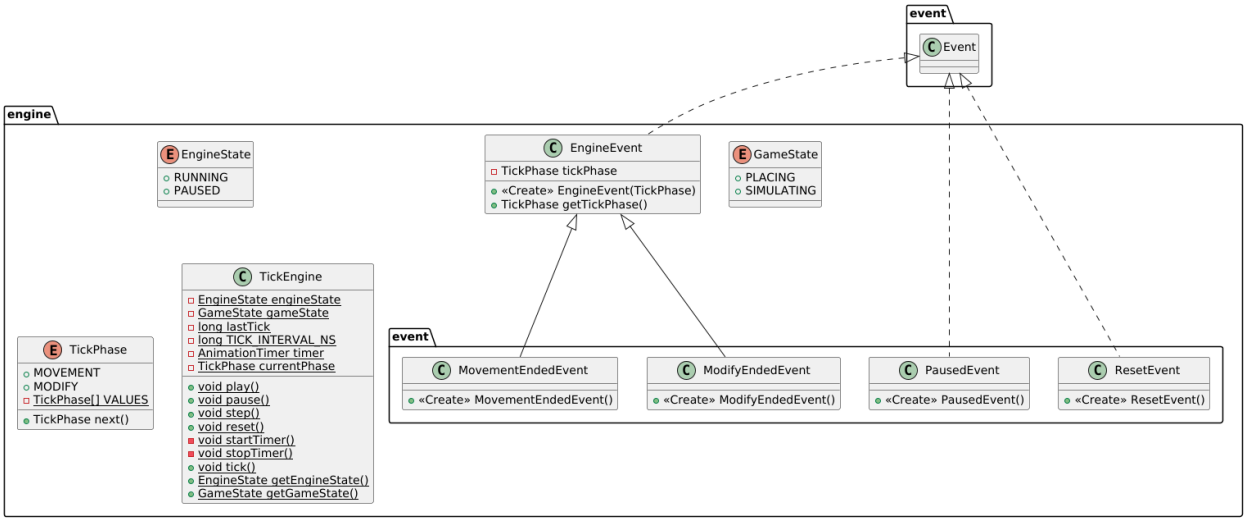

util
GridIndexable
ui
tooltip
Tippable
component
modifier
Modifier
boolean isDisabled
GridPos gridPos
«Create» Modifier()
«Create» Modifier(GridPos)
boolean isDisabled()
void setDisabled(boolean)
boolean checkSetDisable(Card)
boolean checkDestroyGlass(Card)
GridPos getGridPos()
void setGridPos(GridPos)
String toString()
void onSuccess()
void modify(Card)
void reset()
Tooltip getTooltip()
Tooltip getModifierTooltip()
Tooltip getDisabledTooltip()
changer
Changer
T changeValue
ChangeType changeType
«Create» Changer()
T getChange()
void setChange(T)
ChangeType getChangeType()
void change(Card)
boolean isBlocking()
void modify(Card)
void reset()
Tooltip getTooltip()
Tooltip getChangerTooltip()
ChangeType
NUMBER
SUIT
MATERIAL
Setter
Arithmetic
void modify(Card)
SuitSetter
«Create» SuitSetter(Suit)
void change(Card)
Tooltip getTooltip()
ValueSetter
«Create» ValueSetter(int)
void change(Card)
void setChange(Integer)
Tooltip getTooltip()
MaterialSetter
«Create» MaterialSetter(Material)
void change(Card)
Tooltip getTooltip()
Adder
«Create» Adder(int)
void setChange(Integer)
void change(Card)
Tooltip getTooltip()
Subtractor
«Create» Subtractor(int)
void setChange(Integer)
void change(Card)
Tooltip getTooltip()
Multiplier
«Create» Multiplier(int)
void setChange(Integer)
void change(Card)
Tooltip getTooltip()
Divider
«Create» Divider(int)
void setChange(Integer)
void change(Card)
Tooltip getTooltip()



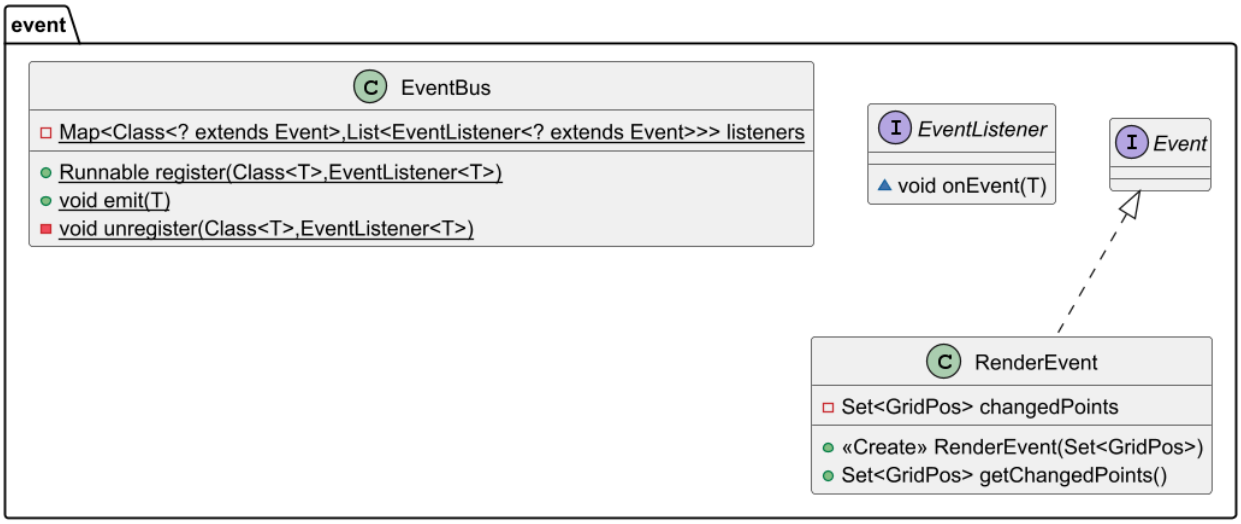
Component.Mover UML



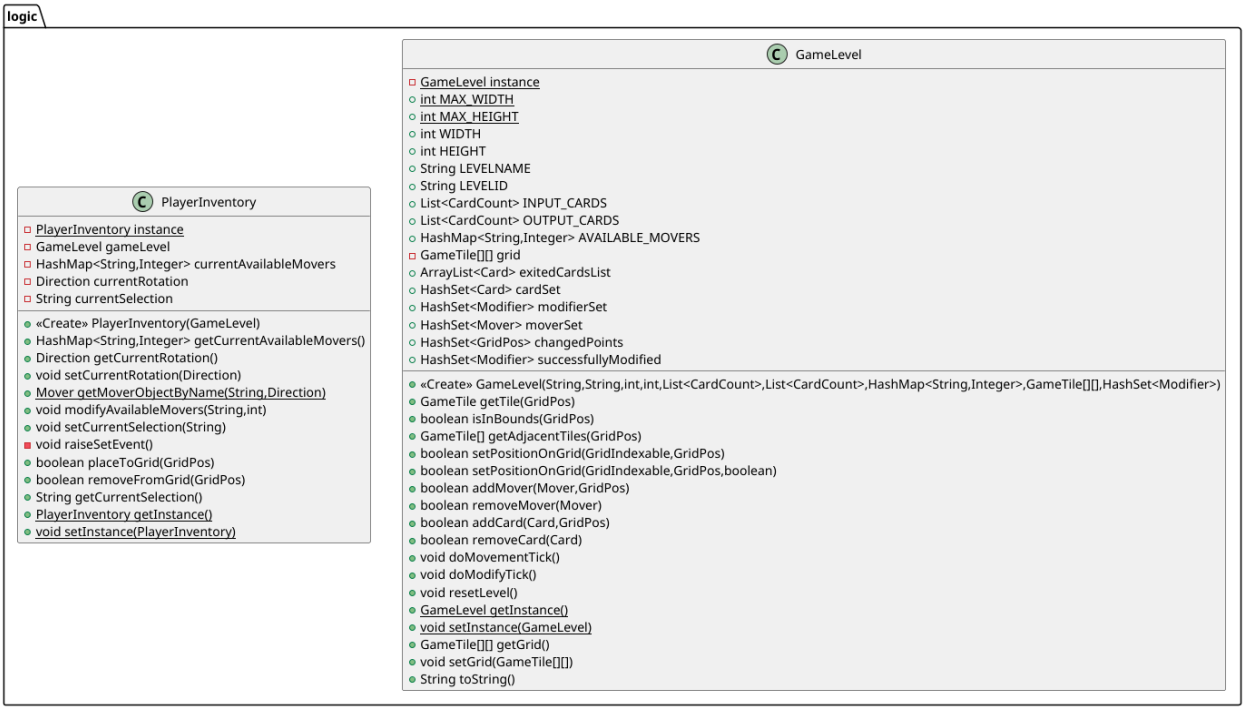
Engine UML

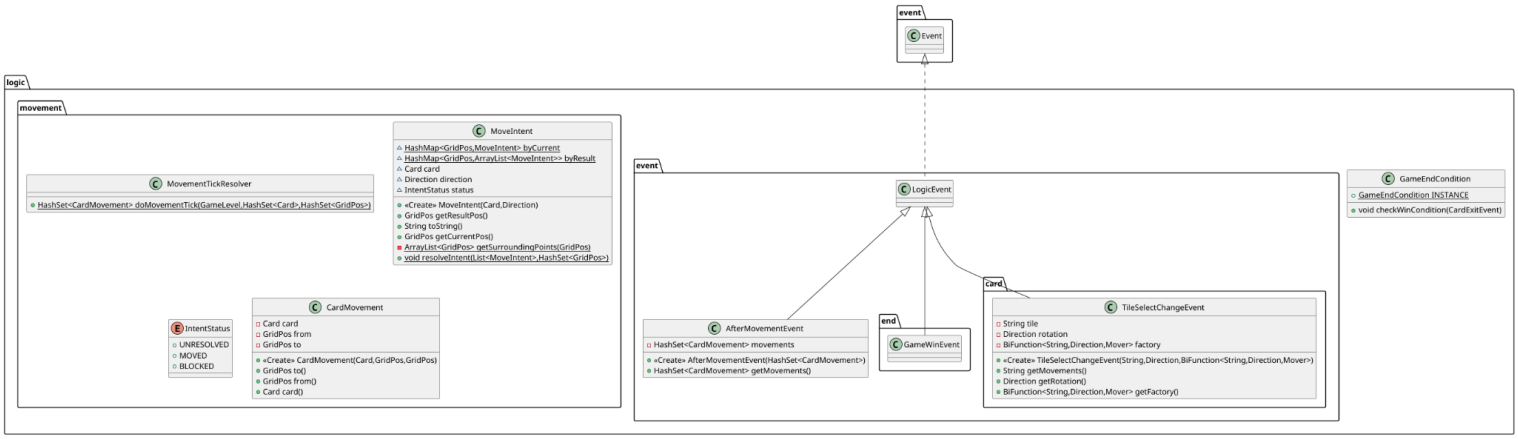


Event UML

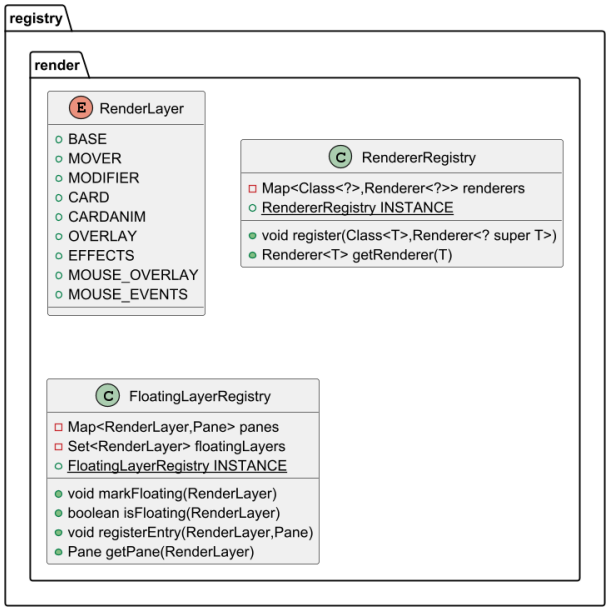


Logic UML

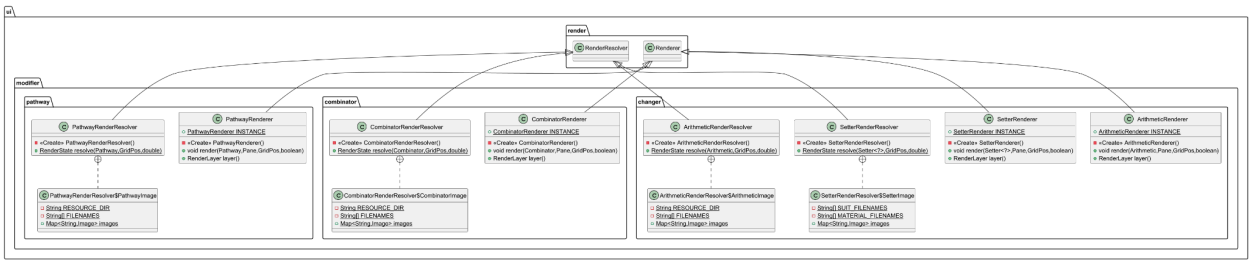

logic
PlayerInventory
PlayerInventory instance
GameLevel gameLevel
HashMap<String,Integer> currentAvailableMovers
Direction currentRotation
String currentSelection
«Create» PlayerInventory(GameLevel)
HashMap<String,Integer> getCurrentAvailableMovers()
Direction getCurrentRotation()
void setCurrentRotation(Direction)
Mover getMoverObjectByName(String,Direction)
void modifyAvailableMovers(String,int)
void setCurrentSelection(String)
void raiseSetEvent()
boolean placeToGrid(GridPos)
boolean removeFromGrid(GridPos)
String getCurrentSelection()
PlayerInventory getInstance()
void setInstance(PlayerInventory)
GameLevel
GameLevel instance
int MAX_WIDTH
int MAX_HEIGHT
int WIDTH
int HEIGHT
String LEVELNAME
String LEVELID
List<CardCount> INPUT_CARDS
List<CardCount> OUTPUT_CARDS
HashMap<String,Integer> AVAILABLE_MOVERS
GameTile[][] grid
ArrayList<Card> exitedCardsList
HashSet<Card> cardSet
HashSet<Modifier> modifierSet
HashSet<Mover> moverSet
HashSet<GridPos> changedPoints
HashSet<Modifier> successfullyModified
«Create» GameLevel(String,String,int,int,List<CardCount>,List<CardCount>,HashMap<String,Integer>,GameTile[][],HashSet<Modifier>)
GameTile getTile(GridPos)
boolean isInBounds(GridPos)
GameTile[] getAdjacentTiles(GridPos)
boolean setPositionOnGrid(GridIndexable,GridPos)
boolean setPositionOnGrid(GridIndexable,GridPos,boolean)
boolean addMover(Mover,GridPos)
boolean removeMover(Mover)
boolean addCard(Card,GridPos)
boolean removeCard(Card)
void doMovementTick()
void doModifyTick()
void resetLevel()
GameLevel getInstance()
void setInstance(GameLevel)
GameTile[][] getGrid()
void setGrid(GameTile[][])
String toString()


event
Event
logic
movement
MovementTickResolver
HashSet<CardMovement> doMovementTick(GameLevel,HashSet<Card>,HashSet<GridPos>)
MoveIntent
HashMap<GridPos,MoveIntent> byCurrent
HashMap<GridPos,ArrayList<MoveIntent>> byResult
Card card
Direction direction
IntentStatus status
«Create» MoveIntent(Card,Direction)
GridPos getResultPos()
String toString()
GridPos getCurrentPos()
ArrayList<GridPos> getSurroundingPoints(GridPos)
void resolveIntent(List<MoveIntent>,HashSet<GridPos>)
IntentStatus
UNRESOLVED
MOVED
BLOCKED
CardMovement
Card card
GridPos from
GridPos to
«Create» CardMovement(Card,GridPos,GridPos)
GridPos to()
GridPos from()
Card card()
event
LogicEvent
AfterMovementEvent
HashSet<CardMovement> movements
«Create» AfterMovementEvent(HashSet<CardMovement>)
HashSet<CardMovement> getMovements()
end
GameWinEvent
card
TileSelectChangeEvent
String tile
Direction rotation
BiFunction<String,Direction,Mover> factory
«Create» TileSelectChangeEvent(String,Direction,BiFunction<String,Direction,Mover>)
String getMovements()
Direction getRotation()
BiFunction<String,Direction,Mover> getFactory()
GameEndCondition
GameEndCondition INSTANCE
void checkWinCondition(CardExitEvent)

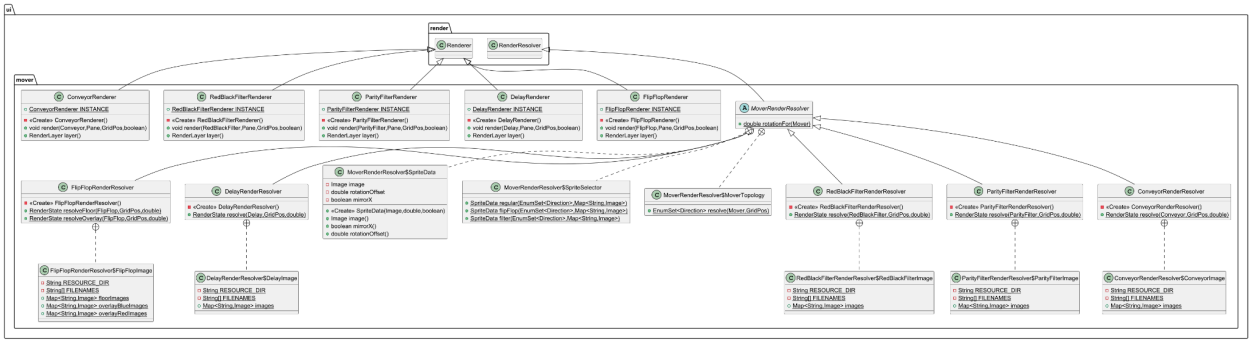
Registry



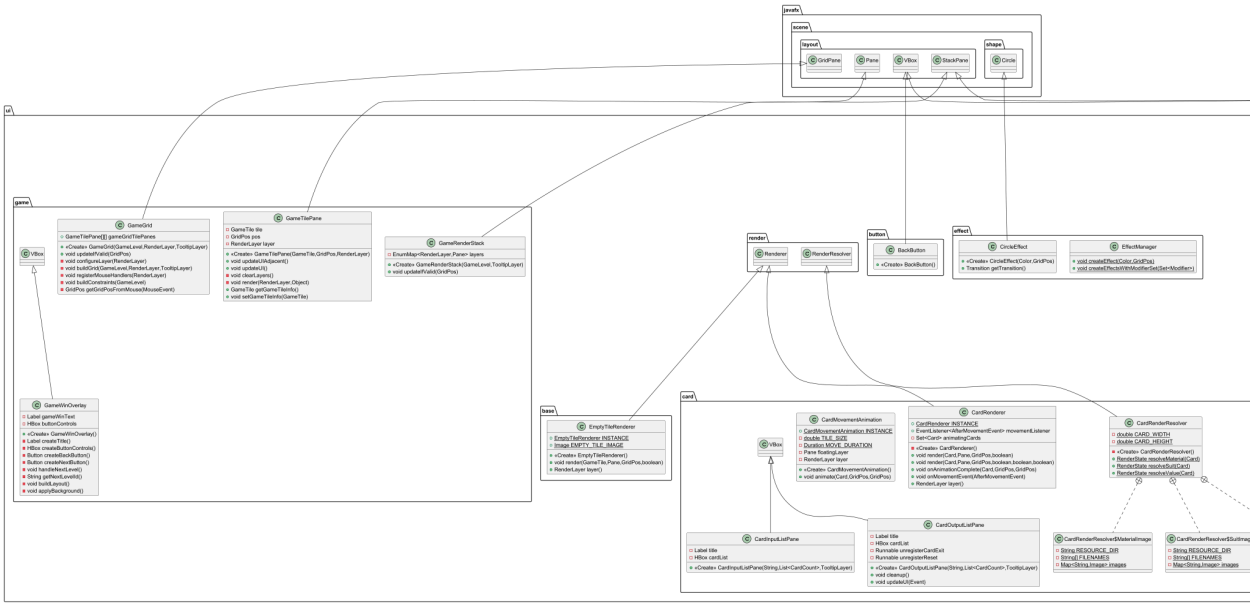
UI.modifier



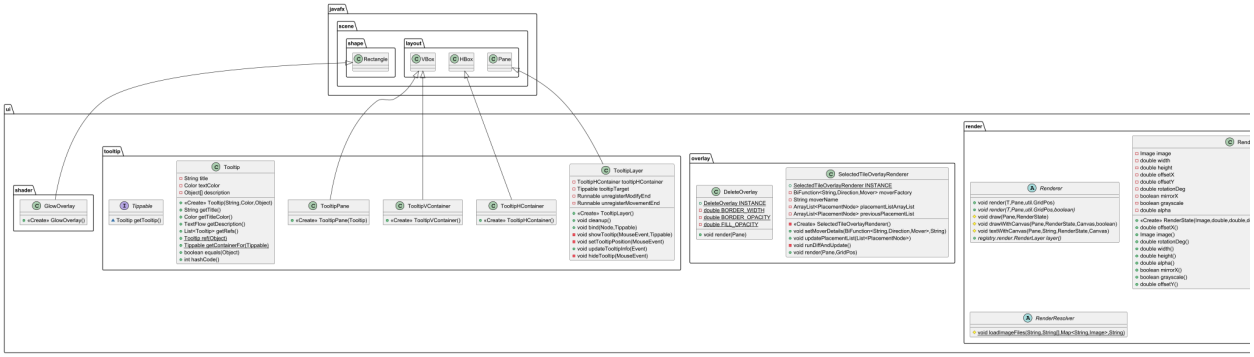
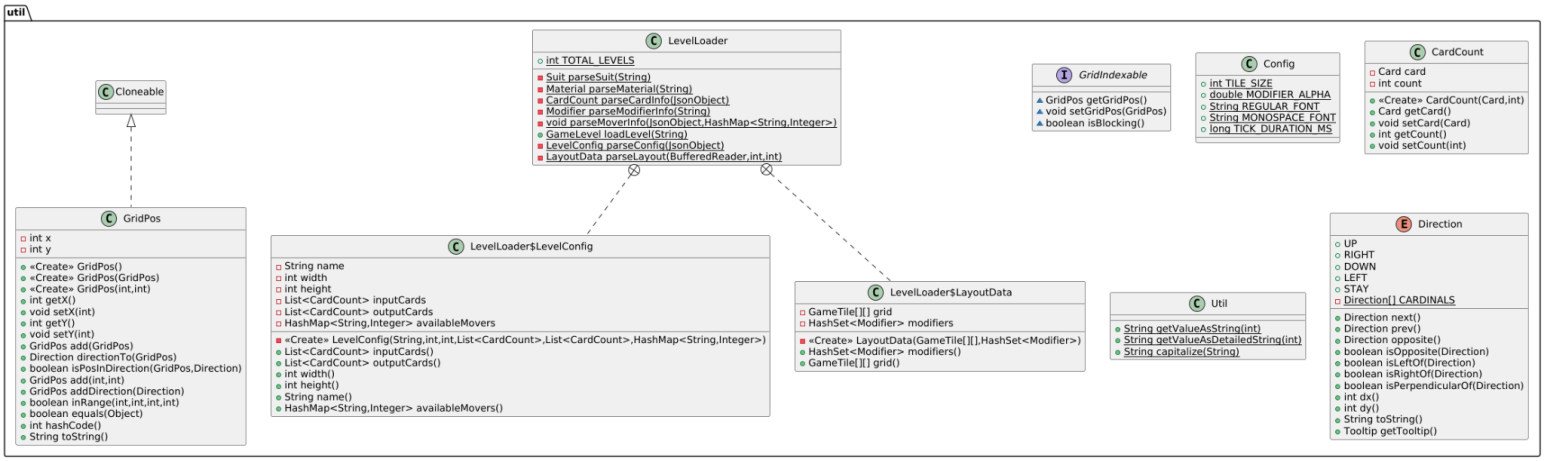
UI.mover



UI.ui1



UI.ui2



Util